



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
INSTITUTO DE CIÊNCIAS EXATAS E DE INFORMÁTICA

Lista de Exercícios 06 - Métodos de Busca

Vitor Costa Oliveira Rolla

Pontifícia Universidade Católica de Minas Gerais

25 de maio de 2026

Questão 01

Enunciado

Sobre os principais conceitos de métodos de busca, responda:

1. Um agente de resolução de problemas precisa ser capaz de formalizar qualquer situação antes de buscar uma solução. Quais são os cinco componentes exigidos para definir um problema de busca em IA? Defina e ilustre cada um com o exemplo do 8-puzzle.
2. Compare os algoritmos de Busca em Largura (BFS) e Busca em Profundidade (DFS) nos critérios de completude e otimalidade. Descreva um cenário ou problema em que a Busca em Profundidade poderia falhar (não terminar).
3. A Busca de Custo Uniforme (equivalente ao algoritmo de Dijkstra) é uma generalização da BFS para grafos com custos variáveis nas arestas. Explique por que a BFS simples não é adequada quando as ações têm custos diferentes. Descreva como a Busca de Custo Uniforme garante encontrar a solução de menor custo. Qual estrutura de dados é fundamental para sua eficiência?
4. O que é uma função heurística $h(n)$ e como ela diferencia a Busca com Informação da Busca sem Informação? Cite pelo menos um exemplo de heurística abordada no material (por exemplo, no mapa da Romênia ou no 8-puzzle).
5. Tanto a Busca Gulosa quanto o A^* utilizam funções heurísticas, mas com estratégias diferentes. Compare os dois algoritmos quanto às suas funções de avaliação $f(n)$, e demonstre com um exemplo por que a Busca Gulosa pode encontrar soluções subótimas enquanto o A^* , com heurística admissível, sempre encontra a solução ótima.
6. Você está desenvolvendo um agente inteligente para três cenários distintos:
 - (a) encontrar o caminho mais curto em um mapa de cidades onde as distâncias em linha reta são conhecidas;
 - (b) jogar xadrez contra um oponente humano com limite de tempo por jogada;
 - (c) resolver um labirinto desconhecido sem nenhuma informação prévia sobre o ambiente.

Para cada cenário, indique qual estratégia de busca seria mais adequada (BFS, DFS, Aprofundamento Iterativo, A^* , MINIMAX com poda alfa-beta) e justifique.

Resolução

1. Um problema de busca é definido por alguns componentes fundamentais: Começa com um estado inicial, e um conjunto de ações possíveis que ele pode executar. O modelo de transição descreve como cada ação leva a novos estados, sendo representado pela função sucessora e, junto com o estado inicial e as ações, forma o espaço de estados. Além disso, há um teste de objetivo, que determina se um estado satisfaz a meta desejada, podendo ser explícito ou baseado em propriedades. Por fim, considera-se o custo da solução, que é a soma dos custos das ações ao longo do caminho, sendo a solução ótima aquela com menor custo total.
2. A busca em largura utiliza uma fila, explorando os nós por níveis e expandindo primeiro os vizinhos mais próximos. Já a busca em profundidade utiliza uma pilha, seguindo um único caminho até o fim antes de retroceder para explorar outras ramificações. No entanto, a DFS pode entrar em loop infinito em grafos com ciclos se não houver controle de estados visitados.
3. Em problemas com custos diferentes nas arestas, a busca em largura não é adequada, pois considera apenas a quantidade de passos, ignorando os pesos e podendo não encontrar o caminho de menor custo. Nesses casos, utiliza-se a busca de custo uniforme, que emprega uma fila de prioridade para expandir o nó de menor custo acumulado, garantindo assim a obtenção da solução ótima.
4. Uma função heurística é uma estimativa que indica o quão próximo um estado está do objetivo, funcionando como um “atalho” que orienta a busca. Com isso, algoritmos informados conseguem explorar caminhos mais promissores primeiro, tornando a busca mais eficiente do que métodos sem informação. Por exemplo a distância em linha reta entre um vértice v qualquer e a cidade de Bucareste (disponibilizado no Slide).
5. A principal diferença está na função de avaliação que cada algoritmo utiliza para escolher o próximo passo. A Busca Gulosa avalia os caminhos usando apenas a estimativa até o objetivo (heurística), enquanto o algoritmo A^* usa a soma dessa estimativa mais o custo real já percorrido para chegar até o nó atual. Desse modo, enquanto o A^* combina o passado e o futuro, ele atualiza os custos dos caminhos encontrados e escolhe o nó com a menor soma do custo real gasto até ali mais a estimativa restante, a busca gulosa olha apenas para o futuro, ou seja, escolhe o nó que parece estar mais perto do objetivo, ignorando o custo já pago, o que faz com que ele não encontre sempre a solução ótima.
6. (a) A^* : Como o objetivo é encontrar o caminho mais curto e se tem as distâncias em linha reta até o destino, pode-se usar essa distância como uma heurística admissível.
(b) MINIMAX com poda alfa-beta: O Minimax simula o jogo. A poda alfa-beta "corta" os ramos inúteis dessa simulação, economizando tempo ao ignorar jogadas que claramente não serão escolhidas.
(c) DFS: Como não há nenhuma informação prévia e o agente precisa descobrir o caminho fisicamente, a DFS simula perfeitamente o comportamento de um explorador em um labirinto: ele escolhe um caminho e vai até o fim, se bater no muro ele volta pelo mesmo caminho até a última bifurcação e tenta a próxima opção.

Questão 02

Enunciado

Considere o problema de busca definido por um grafo, onde A é o estado inicial e G é o estado objetivo, com custos nas arestas.

Resolva utilizando:

- Busca em largura (BFS)
- Busca em profundidade (DFS)
- Busca de custo uniforme
- Busca gulosa
- A*

Para cada estratégia:

- Mostre a árvore de busca final
- Indique o caminho encontrado de A até G
- Calcule o custo total da solução

Considere ordem de visitação alfabética.

Resolução

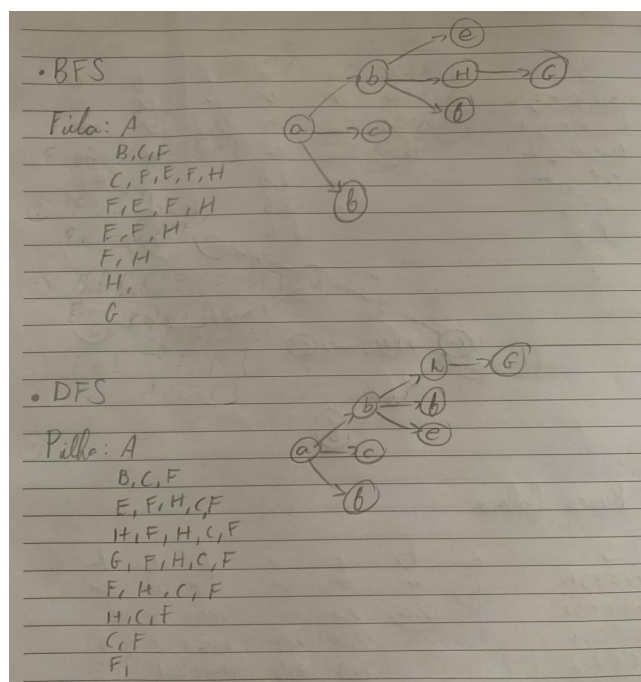


Figura 1: Primeira parte da resolução.

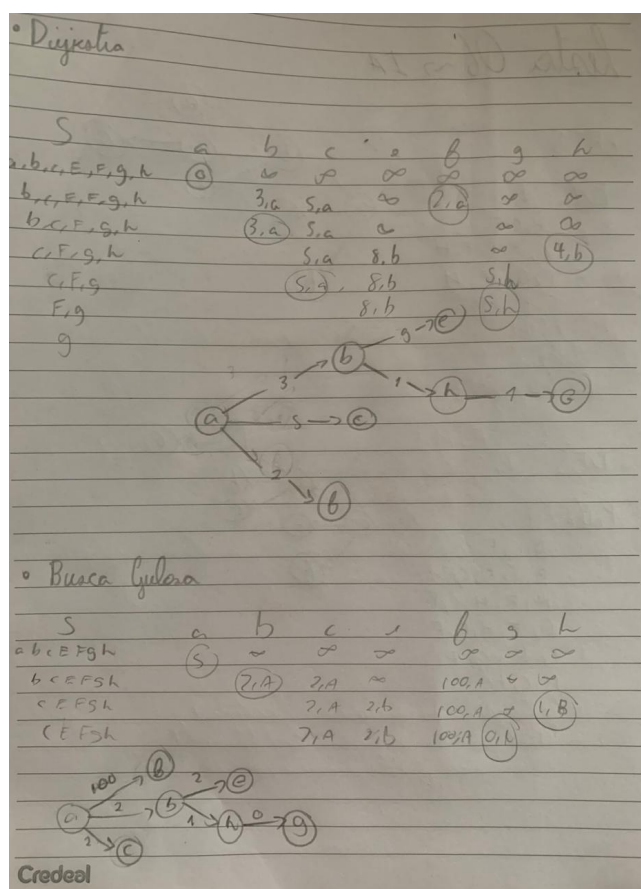


Figura 2: Segunda parte da resolução.

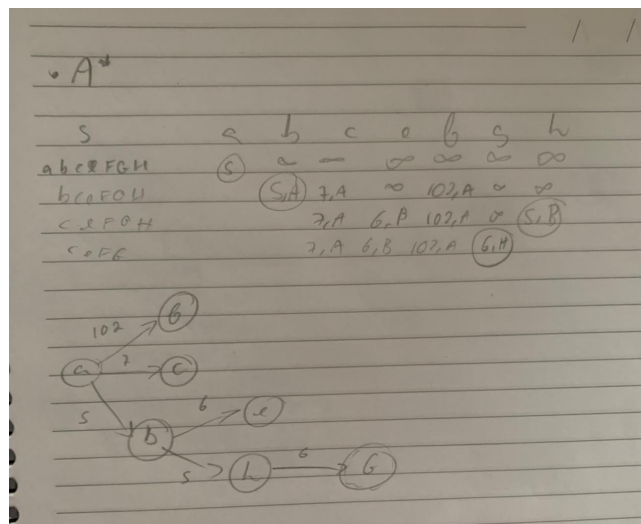


Figura 3: Terceira parte da resolução.

Questão 03

Enunciado

Considere uma árvore de jogo de soma zero:

- Nós MAX: triângulos para cima
- Nós MIN: triângulos para baixo
- Folhas: valores de utilidade

Aplique o algoritmo MINIMAX:

- Determine a melhor ação para o jogador MAX
- Mostre a árvore com os valores minimax em cada nó

Resolução

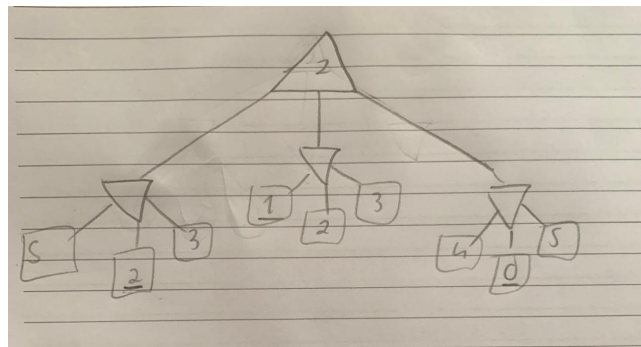


Figura 4: MINMAX Aplicado